

임채현

BACKEND DEVELOPER

AI를 어디까지 쓸지 판단하는 개발자

문제를 먼저 정의하고, LLM에 맡길 부분과 코드로 확정할 부분의 경계를 나눈 뒤 구현합니다. 판단마다 기각한 대안과 트레이드오프를 남기고, 변경은 골든셋·회귀 테스트로 검증합니다.

EMAIL chaehyun010104@gmail.com

PORTFOLIO https://chaehyunlim-portfolio.vercel.app

GITHUB https://github.com/Chaehyunli

기술 블로그 https://ch010104.tistory.com/

강점과 근거

LLM·알고리즘 경계 설계

프롬프트로 지켜지지 않는 조건은 코드로 확정하고, 규칙으로 못 잡는 판단만 LLM에 남긴다.

- 모두약속 — DeepEval 19개 테스트 검증 — p.3
- Masil — 테스트 요청 24건 모두 정확히 분류 — p.5

인증·신뢰 경계 설계

데이터가 저장되는 시점과 쓰이는 시점을 나눠, 서버가 무엇을 볼 수 있는지부터 정한다.

- 모두약속 — 서버도 키를 풀 수 없는 구조로 재설계 — p.3
- PETNER — WebSocket 세션 인증 — p.8

검색·파이프라인 최적화

병목을 연산과 통신, DB와 외부 호출로 나눠 측정한 뒤 그 구간만 고친다.

- Searchive — 왕복 5→1회 · 응답 250→10ms — p.7
- Masil — k6 동시 접속 100명 실패 없이 처리 — p.5
- 모두약속 — 네이버 검색 호출 36→21회 — p.3

CAREER

2026.01 ~ 2026.02

프람트테크놀로지 · 개발팀 인턴 — 레거시 → MSA 전환 품질 검증

각자 맡은 모듈만 보던 방식에 의문을 갖고 레거시 코드를 공통 기준으로 약 20만 줄, 4개 모듈을 함께 검증했다.

8,000개 단위 테스트 설계

100% 인턴 기간 내 오류 해결률

p.2

SKILLS

LANGUAGES	Java · Python · TypeScript · SQL
BACKEND	Spring Boot · Spring Security · Spring WebFlux / R2DBC · FastAPI · JPA / SQLModel · WebSocket·STOMP · SSE
DATA · SEARCH	PostgreSQL · MySQL · Redis · Elasticsearch / OpenSearch · pgvector · Alembic / Flyway
AI 파이프라인	PydanticAI · DeepEval 골든셋·회귀 fixture · RAG · KeyBERT · XGBoost
INFRA	Docker Compose · Vercel · Render · Git

CAREER

2026.01 ~ 2026.02

Spring Boot · Thymeleaf · MyBatis · MSA

프람트테크놀로지 · 개발팀 인턴 — 레거시 → MSA 전환 품질 검증

약 15명이 약 20만 줄 규모의 전략물자 관리 시스템을 4개 모듈(portal·permission·judgement·destination) MSA로 옮기는 프로젝트에서 레거시 분석과 테스트 설계를 맡았다.

8,000개
단위 테스트 설계·수행

100%
인턴 기간 내 오류 해결률

4개
함께 검증한 MSA 모듈

모듈 단위 개발에서 전 모듈 검증으로
각자 맡은 모듈만 보는 방식으로는 모듈 간 표준을 세울 수 없고 요구사항 누락도 생긴다고 판단해, 레거시 코드를 공통 기준으로 모듈별 현황과 요구사항부터 정리했다.

필터 유지 요구를 방식 통일로 연결
검색 필터가 query 파라미터와 request body로 섞인 것을 발견했고, 관리원이 목록 복귀 시 필터 유지를 요청하자 조건이 URL에 남는 query 파라미터로 통일을 제안했다. 개발팀장 확인을 거쳐 전 모듈에 반영됐다.

오류는 원인 관점으로 공유
레거시와의 차이·재현 조건·발생 모듈 관점으로 정리해 공유했고, 찾은 오류는 인턴 기간 안에 모두 수정됐다.

EDUCATION & TRAINING

2021.03 ~ 2026.08
학력
명지대학교 컴퓨터공학부 컴퓨터공학과 졸업

2025.07 ~ 2025.08
교육
SK AI Dream Camp 중급

2026 ~ 현재
교육
SKALA 4기 — Agile/Scrum·MSA 팀 실습(SM), 응답 계약 기반 병렬 개발

자격
정보처리기사 · SQLD

PERSONAL AI

2026.07 ~ 현재

HERMES · 개인 프로젝트 · 설계·구축·운영 단독

대화로 정한 관심사·규칙을 기억해 뉴스·채용·학습 업무를 자동화하는 개인 AI 업무 환경

Hermes Agent · Linux VPS · Discord · MCP · Notion · Obsidian · Cron · Python · Hook

92.1%
최근 30일 684건 중 630건 완료

10개
VPS에서 상시 운영하는 Cron 자동화

3개
LLM 없이 스크립트로 실행하는 자동화

AI 판단 오류를 규칙으로 보정
관심사·우선순위와 어긋날 때마다 대화 피드백을 메모리·Skill에 반영하고, 반복 오판은 Hook 규칙으로 전환했다.

스크립트와 AI의 역할 분리
규칙이 명확한 수집은 스크립트로 실행하고 요약·선별만 AI에 맡겼다. 원본 검증이 실패하면 Notion 쓰기를 멈추고 원인을 기록한다.

PROJECTS 4개 상세 · 2개는 웹 포트폴리오에서

2026.08 ~ p.3
모두약속 배포 운영 중
목적·시간·지역·예산·선호를 입력하면 실제 장소 검색 결과로 약속 일정을 만들어주는 AI 서비스

2026.03 ~ 2026.08 p.5
Masil 앱 배포 완료
대화로 일정을 짜고 예약까지 이어지는 AI agent 여행 플래너

2025.10 ~ 2025.12 p.7
Searchive 완성
문서를 업로드하면 자동 태깅·검색·RAG 질의응답으로 이어지는 개인 지식 베이스

2025.08 ~ 2025.10 p.8
PETNER 장려상
유기견 탐색·입양 신청·커뮤니티·보호소 실시간 채팅을 연결한 팀 백엔드 서비스

2025.01 ~ 2025.03 웹에서 보기 ↗
동아리모아 완성
동아리 탐색·지원·운영·권한 위임을 한 서비스에서 다룬 팀 백엔드 프로젝트

2025.03 ~ 2025.10 웹에서 보기 ↗
노소공 완성
행동 데이터로 펫 감정을 예측하고 미니게임 보상·성장 흐름을 연결한 ML 기반 동물 육성 게임

모두약속

배포 운영 중

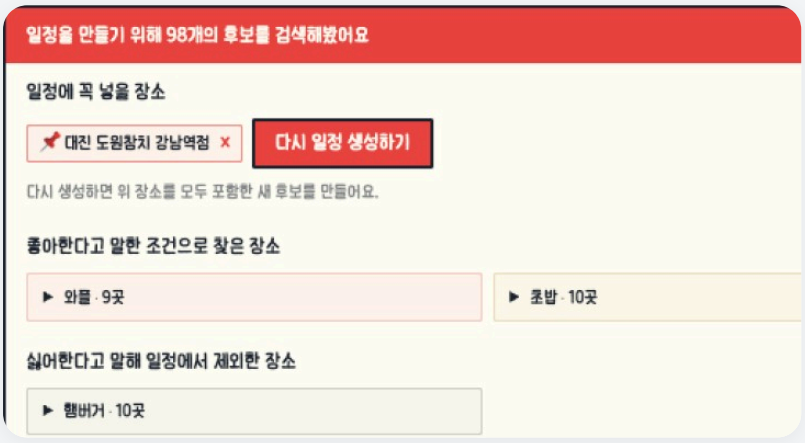
목적·시간·지역·예산·선호를 입력하면 실제 장소 검색 결과로 약속 일정을 만들어주는 AI 서비스

기간·형태	2026.08 ~ · 개인 프로젝트
맡은 역할	기획·설계·구현·배포 단독
TECH	FastAPI · SQLModel · PostgreSQL · Redis · Vue 3 · TypeScript · Web Crypto API · DeepEval
LINKS	배포 https://moduyaksok.vercel.app · GITHUB https://github.com/Chaehyunli/moduyaksok · 상세 https://chaehyunlim-portfolio.vercel.app/moduyaksok

1개월
기획부터 배포까지 단독
실제 약속에 활용 중

19개
DeepEval 테스트로 검증
GEval 임계값 0.70

60만 회
PBKDF2 키 유도
서버도 키를 풀 수 없는 구조



실제 검색한 장소 후보 — 일정은 이 안에서만 구성한다

배경

AI가 실재하지 않는 장소를 추천하거나 실제 이동거리를 반영하지 못해 일정이 틀어지는 한계가 있었다. 그래서 실제 검색 결과 안에서만 일정을 구성하고, 실제 경로로 동선을 검증하는 서비스를 혼자 한 달 만에 기획부터 배포까지 진행했다.

설계



판단 01

AI가 제안한 키 보관 구조를 검증해, 서버도 키를 풀 수 없게 재설계했다

문제	BYOK 키 보관 구조를 Manyfast AI로 초안 설계했는데, 서버가 마스터 키로 모든 사용자 키를 복호화할 수 있어 서버가 침해되면 등록된 키 전체가 노출될 위험을 발견했다. 현업 개발자 자문을 받아 편의성과 신뢰성의 균형을 기준으로 삼았다.
기각한 안	클라이언트 직접 호출(구현 비용) · 요청마다 키 전달(평문 노출) · localStorage 저장(XSS 유출)
선택	• 비밀문구를 PBKDF2로 60만 회 반복해 만든 키로 브라우저에서 AES-GCM 암호화하고, 서버에는 암호문만 저장 • 비밀문구 원문은 버리고 만든 키만 sessionStorage에 두어 비밀문구는 한 번만 입력
결과	**서버도 사용자 키를 풀 수 없는 구조**로 바꾸고 비밀문구는 **한 번만 입력**하게 해 신뢰성과 편의성의 균형을 맞췄다.
트레이드오프	비밀문구를 잃으면 키를 복구할 수 없어 삭제 후 재등록으로 대응한다.

판단 02

LLM은 후보를 구성하고, 조건 보장은 알고리즘이 맡게 나눴다

문제	"태그당 후보 1곳", "식사 시간대엔 식사 장소" 같은 지시를 프롬프트로만 맡기자 지켜지지 않았고, 후보가 비슷한 장소로 수렴하거나 동선이 비현실적인 문제가 반복됐다.
판단 기준	시간·예산·이동거리·식사 슬롯·태그 중복은 코드가 확정할 수 있다 — 규칙으로 못 잡는 의미 판단만 LLM에 남긴다
선택	• 조건 정규화 → 후보 생성 → 검증·병합 → 동선 보강 4단계로 나누고, 규칙 하드 필터를 통과한 후보에만 LLM을 1회 호출 • LLM이 맡은 단계는 DeepEval로, 코드가 맡은 조건은 회귀 테스트로 검증
결과	수렴·취향·동선 **세 가지 문제를 모두 해결**하고, 테스트를 모두 통과한 뒤 배포해 안정적으로 운영하고 있다.

함께 한 판단

외부 API 호출량 제어

요약	네이버 검색의 초당 호출은 요청끼리 번갈아 쓰는 라운드로빈으로, 일일 한도는 Redis 카운터로 제어했다. 예산이 부족하면 차단 대신 확보 가능한 만큼만 검색했고, 하이브리드 전환으로 일정 1건당 네이버 검색 호출을 **36회에서 21회**로 줄였다.
남은 과제	초당 리미터가 서버 프로세스 안에서만 동작해, 워커를 늘리면 전역 상한이 어긋난다.

Masil

앱 배포 완료

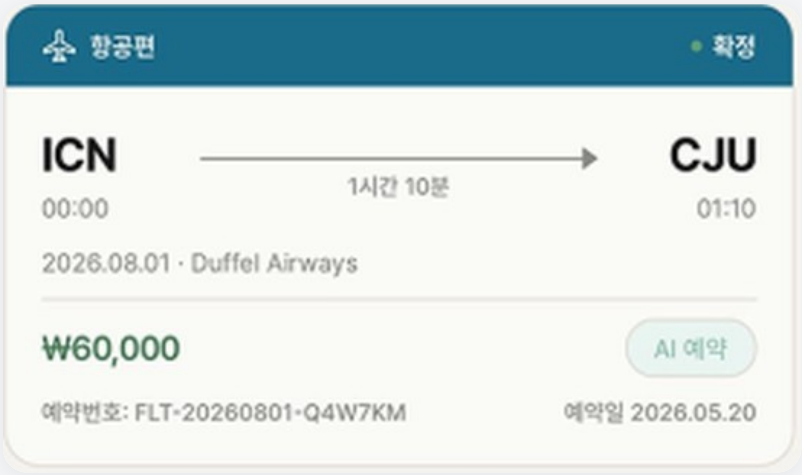
대화로 일정을 짜고 예약까지 이어지는 AI agent 여행 플래너

기간·형태	2026.03 ~ 2026.08 · 4인 팀
맡은 역할	**AI 파이프라인 설계 단독 · AI 서버 구현 · 백엔드 API 설계·일정 도메인 구현 · R2DBC 비동기 전환** 팀원 4명 모두 AI 개발이 처음인 상황에서 자원해 에이전트 오케스트레이션을 독학하며 AI 서버를 맡았다.
TECH	React Native · Spring WebFlux · R2DBC · FastAPI · PydanticAI · PostgreSQL · Redis · SSE · DeepEval
LINKS	GITHUB https://github.com/orgs/Masil2026/repositories 담당 PR GitHub에서 담당 PR 목록 보기 ↗ 상세 https://chaehyunlim-portfolio.vercel.app/masil

24/24
테스트 요청 정확히 분류
DeepEval · 3개 시나리오 0.70~0.80

100명
동시 접속 실패 없이 처리
k6 · p95 11.1초(기준 12초 미만)

은상
명지대 캡스톤디자인 경진대회
26개 팀 중 · 원스토어 출시

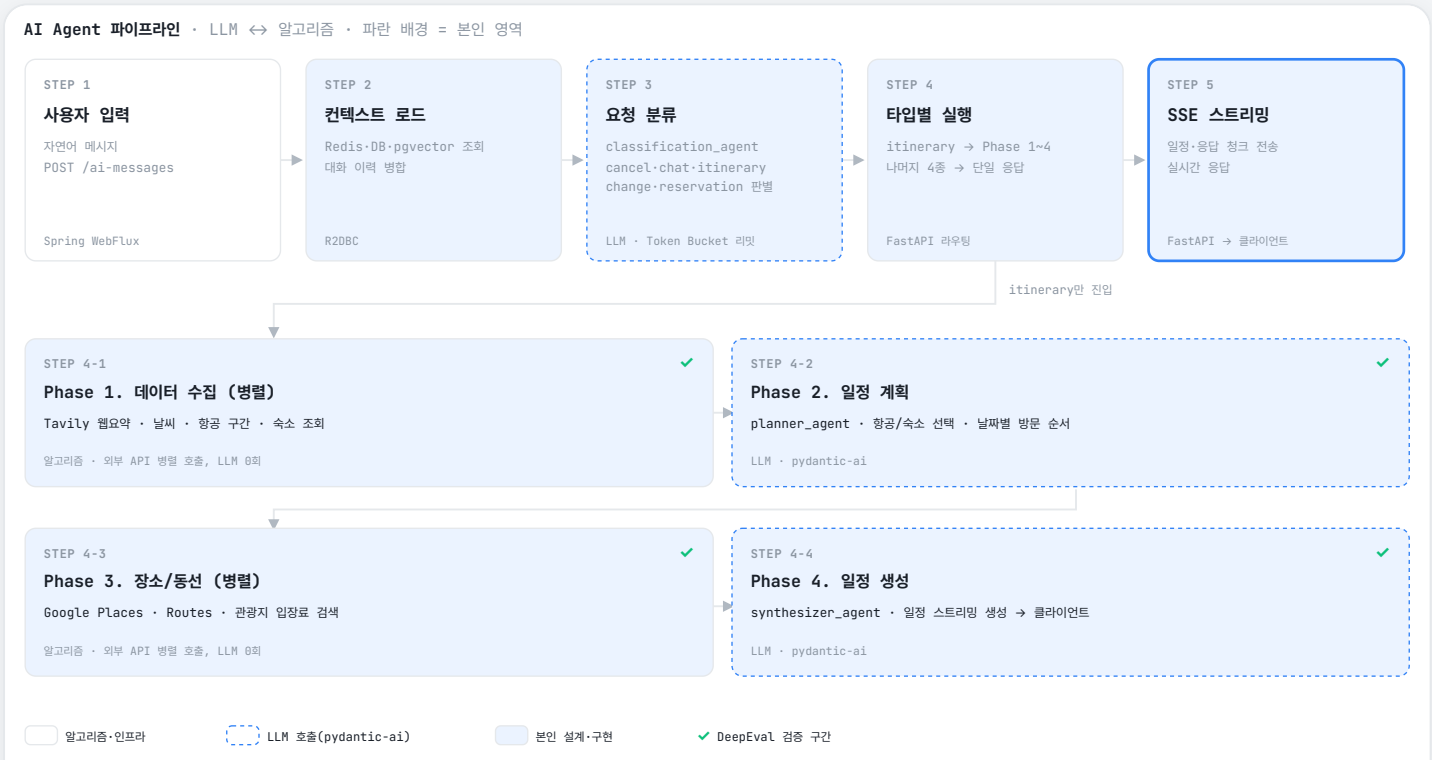


예약 상태 — 잘못 취소되면 되돌릴 수 없는 화면

배경

기존 ML 기반 여행 서비스는 일정만 제안할 뿐 실시간 항공·숙소·교통 정보를 반영하지 못했다. 대화로 일정부터 예약까지 처리하는 AI 에이전트를 기획하고, 예약까지 믿고 맡길 수 있는 에이전트를 목표로 삼았다.

설계



판단 01

되돌릴 수 없는 동작은 파이프라인 진입 전에 확정했다

문제	"그거 취소해줘"가 일정 재생성인지 예약 취소인지 구분되지 않아 되돌릴 수 없는 예약이 잘못 실행됐다. 분류와 응답 생성이 한 단계에 섞여 있어 어디서 틀렸는지도 찾을 수 없었다.
판단 기준	파이프라인 안에서 분류하면 이미 실행이 시작된 뒤다 — 진입 전에 판단해야 한다
선택	• classification_agent가 의도를 5가지(대화·일정 생성·예약·수정·취소)로 먼저 분류하고, 애매하면 실행 대신 확인 단계로 분기 • 분류·계획·합성 단계마다 DeepEval 평가를 붙여 틀린 단계를 바로 특정
결과	테스트 요청 **24건을 모두 정확히 분류**하고 3개 시나리오 모두 GEval 0.70~0.80을 확인한 뒤 원스토어에 출시했다.

판단 02

블로킹으로 남은 DB 접근을 비동기 전환과 호출량 제어로 풀었다

문제	WebFlux로 요청을 비동기로 받았지만 DB 접근은 블로킹 JPA였다. 느린 AI 호출이 몰리면 요청이 DB 커넥션을 쥔 채 기다려 적은 접속자에도 요청이 밀렸고, LLM API 429 에러도 잦았다.
판단 기준	R2DBC는 커넥션 수를 늘리는 게 아니라 붙잡는 시간을 줄인다 — 같은 15개로 더 많은 요청을 처리
선택	• 은상 수상 후 R2DBC를 직접 학습해 데이터 접근 계층까지 비동기로 전환 • Token Bucket으로 LLM 호출 빈도를 제어하고, 429 발생 시 지수 백오프로 재시도
결과	DB 커넥션 풀 15개 환경에서 k6로 **동시 접속 100명까지 실패 없이** 처리했고(p95 11.1초, 기준 12초 미만), **110명부터 지연**이 발생하는 한계선도 확인했다.

Searchive

완성

문서를 업로드하면 자동 태깅·검색·RAG 질의응답으로 이어지는 개인 지식 베이스

기간·형태	2025.10 ~ 2025.12 · 개인 프로젝트
맡은 역할	기획·설계·구현 단독
TECH	FastAPI · PostgreSQL · Elasticsearch · pgvector · Redis · MinIO · KeyBERT · React
LINKS	GITHUB https://github.com/orgs/Searchive-Project/repositories 상세 https://chaehyunlim-portfolio.vercel.app/searchive

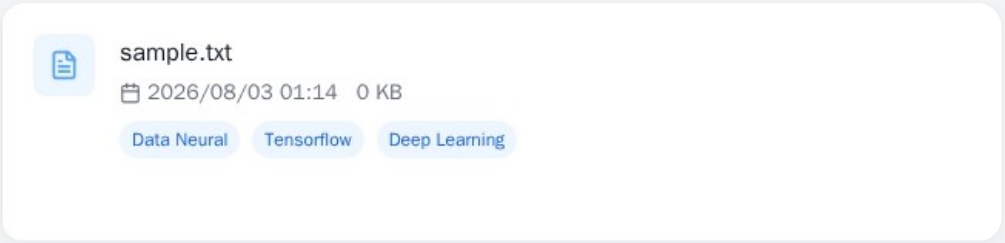
5→1회
Elasticsearch 네트워크 왕복
키워드 5개 기준

250→10ms
자동 태깅 응답 시간
96% 단축

3배
목표치 대비 태그 후보 과추출
필터 뒤 태그 0개 방지

배경

문서가 쌓일수록 필요한 자료를 찾기 어려워진다. 외부 서비스 없이 업로드부터 자동 태깅·검색·RAG 질의응답까지 이어지는 개인 지식베이스를 두 달간 혼자 기획·설계·구현했다.



업로드 직후 정제된 태그가 붙은 문서

설계



판단 01

계산과 통신이 함께 N배로 늘어나는 구조를 배치 검색으로 풀었다

문제	키워드 후보마다 Elasticsearch에 개별 요청을 보내, 키워드 5개면 자동 태깅 응답이 250ms까지 늘어났다.
판단 기준	순차·배치 방식의 시간을 따로 측정해, 원인이 연산이 아니라 네트워크 왕복 횟수임을 확인
선택	• 후보별 KNN 쿼리는 그대로 두고 _msearch 요청 한 번으로 묶어 전송 • 응답을 원래 후보 순서로 복원해 이후 판단 로직은 바꾸지 않음
결과	키워드 5개 기준 네트워크 왕복을 **5회에서 1회**로 줄이고, 응답 시간을 **250ms에서 10ms**로 96% 단축했다.

판단 02

AI가 뽑은 태그 후보를 그대로 저장하지 않았다

문제	KeyBERT 후보에 불용어·숫자·한 글자 토큰이 섞여 필터링하면 태그가 0개가 되기도 했고, 표기만 다른 같은 개념이 여러 태그로 흩어졌다.
기각한 안	목표 개수만 추출(거를 여유 없음) · 완전 일치만 사용(표기 차이 못 흡수) · 유사도만 사용(다른 의미를 합칠 위험)
선택	• 태그 후보를 목표치의 3배로 과추출한 뒤 불용어·숫자·한 글자 토큰을 거르고 표기를 정규화 • 완전 일치 태그를 먼저 재사용하고, 없으면 pgvector 유사도 0.8 이상인 대표 태그로 병합
결과	필터 뒤 **태그가 0개가 되는 문제를 막고**, 사용자가 보는 태그를 일관되게 유지했다.

PETNER

장려상

유기견 탐색·입양 신청·커뮤니티·보호소 실시간 채팅을 연결한 팀 백엔드 서비스

기간 · 형태	2025.08 ~ 2025.10 · 4인 팀
맡은 역할	**채팅·유기견·입양 신청·즐겨찾기 도메인 백엔드 · API·ERD 단독 설계**
성과	**명지대 2025 Cursor AI 기반 VIBE CODING 실전활용 경진대회 본선 8팀 중 장려상**
TECH	Java 17 · Spring Boot · Spring Security · JPA · PostgreSQL · Redis · OpenSearch · WebSocket/STOMP
LINKS	GITHUB https://github.com/orgs/Dangdaengdan/repositories · 담당 PR GitHub에서 담당 PR 목록 보기 ↗ 상세 https://chaehyunlim-portfolio.vercel.app/petner

WebSocket 세션 인증

로그인 세션 인증이 REST와 WebSocket이라는 서로 다른 경계를 넘어도 끊기지 않도록 STOMP 핸드셰이크 구조를 설계하고, 채팅방 생성부터 이전 메시지 페이징까지 이 위에서 확장했다.

채팅방 나가기 = Soft Delete

참여 레코드를 비활성화해 FK 무결성을 지키고, 입장·퇴장 시각으로 메시지 범위를 나눠 재입장한 사람에겐 이후 메시지만, 보호소에겐 상담 이력이 남게 했다.

검색엔진 선택

오타 허용 검색이 필요해 Elasticsearch와 OpenSearch의 성능과 라이선스를 비교했고, 가볍고 Apache 2.0으로 제약이 적은 OpenSearch가 충분하다고 판단해 채택했다.